

C#.NET



Multithreading

Part-7

Mr. Bangar Raju

 **NARESH**
technologies

Understanding the Performance Benefits of Multithreading in C#

By Albert Harmon

Published April 15, 2017

🕒 3 min read

[Technology](#)



TLDR This blog post explores the performance advantages of multithreading over single-threaded models in C#. It demonstrates through code examples how multithreading can significantly reduce

execution time for tasks, making applications more efficient.

In this blog post, we will explore the concept of multithreading in C# and how it can enhance the performance of applications compared to single-threaded models. We will demonstrate this through practical code examples and performance measurements.

Multithreading is a programming technique that allows multiple threads to run concurrently, enabling more efficient use of resources and faster execution of tasks. In contrast, a single-threaded model processes tasks sequentially, which can lead to longer execution times, especially for resource-intensive operations.

Setting Up the Experiment

To illustrate the performance differences between single-threaded and multi-threaded models, we will create a simple application that increments a counter. We will implement this in both models and measure the execution time for each.

Creating the Counter Class

First, we will create a new class named `ThreadPerformance``. In this class, we will define a method to increment a counter:

```
using System;
using System.Diagnostics;

public class ThreadPerformance
{
    public void IncrementCounter()
    {
        long count = 0;
        for (long i = 0; i <= 100000000; i++)
        {
            count++;
        }
        Console.WriteLine("Value of count: " + count);
    }
}
```

Measuring Execution Time in Single-Threaded Model

Next, we will measure the execution time of the `IncrementCounter` method in a single-threaded model:

```
class Program
{
    static void Main(string[] args)
    {
        Stopwatch stopwatch = new Stopwatch();
```

```

        ThreadPerformance performance = new ThreadPerformance(

        stopwatch.Start();
        performance.IncrementCounter();
        stopwatch.Stop();

        Console.WriteLine("Time taken in single-threaded model
    }
}

```

When we run this code, we might observe that it takes a significant amount of time to complete the task, for example, around 472 milliseconds for 100 million increments.

Measuring Execution Time in Multi-Threaded Model

Now, let's implement the same functionality using multithreading. We will create two threads to run the `IncrementCounter` method concurrently:

```

class Program
{
    static void Main(string[] args)
    {
        Stopwatch stopwatch = new Stopwatch();
        ThreadPerformance performance = new ThreadPerformance(

```

```
Thread thread1 = new Thread(new ThreadStart(performanc
Thread thread2 = new Thread(new ThreadStart(performanc

stopwatch.Start();
thread1.Start();
thread2.Start();

thread1.Join();
thread2.Join();
stopwatch.Stop();

Console.WriteLine("Time taken in multi-threaded model:
}
}
```

In this case, the execution time is drastically reduced, potentially down to just a few milliseconds, demonstrating the efficiency of multithreading.

Performance Comparison



When we compare the execution times:

- **Single-threaded model:** Approximately 472 milliseconds for 100 million increments.
- **Multi-threaded model:** As low as 3 milliseconds for the same task.

This stark contrast highlights the performance benefits of using multithreading in applications.

Conclusion

The results of our experiment clearly show that multithreading can significantly improve application performance by utilizing CPU resources more effectively. This is particularly important in GUI applications, where the main thread can remain responsive to user interactions while background tasks are processed in separate threads.

In summary, multithreading is a powerful tool in C# that allows developers to create more efficient applications. By leveraging multiple threads, we can reduce execution time and enhance the overall user experience.

Thank you for reading this blog post. For more insights and tutorials, stay tuned for our upcoming content.